Ali's Agentic Penetration-Testing Framework

Physical & logical topology + end-to-end workflow. All execution components live on the lab host (`lab-host.local`) — the agent runtime container mounts the host docker.sock, so its "containers" are the same machines. Rendered from the live `agentic-pentest-pipeline` skill tree, `pipeline-build.md`, and host `docker ps`.

LAYER 1 Physical / Logical Topology

A — Telegram Bot

@ali_agent_90bot — entry point. Authorization intent + commands land here.

`gate: bot-token env var + user-ID allowlist`

B — Agent Runtime (container)

Orchestrator + agent loop. Parses auth, runs passive recon, launches the agent pass.

`containerized: agent runtime (API + dashboard)`

C — Lab Host

Substrate: sandboxed execution user.

`lab-host.local`

D — LiteLLM Router

Model routing for claude/codex backends.

`docker: llm-router :4000`

E — Model Gateway (hosted)

Anthropic + chat/completions model provider.

`Anthropic + OpenAI-compatible APIs`

F — PentestGPT Agent (unified_agent harness)

Autonomous tester. Backends: **claude** (→ gateway /v1/messages → deepseek-v4-flash→mimo), **codex** (OpenAI /v1/responses), **openai-chat** (→ gateway /v1/chat/completions → hy3 / ox-alpha-free). Supervisor spawns executor turns that fire shell commands at the target.

`~/PentestGPT/pentestgpt_agent (uv venv) · runs/<trial>/`

G — Scanner Toolchain

ZAP (bounded, docker :8092) · nuclei (binary) · sqlmap (staging net) · nmap (host) · whatweb (host) · exploit-db (local GitLab clone, searchsploit) for version→PoC correlation. Fired by the agent's shell AND the scanner pass.

`ZAP:8092 · nmap/whatweb on host · sqlmap in staging net`

H — mitmproxy capture

HTTPS capture + evidence addon; handles Cloudflare TLS where ZAP's JDK fails.

`docker: mitm :8083`

I — SPA Crawler

Playwright headless. Closes the SPA blind spot — extracts routes + every XHR/fetch.

`pipeline/pentest-crawler (venv)`

J — Evidence Store

Content-addressed SQLite + blobs. Single source of truth for findings/tool-runs.

`pipeline/evidence/evidence.db + blobs/`

K — Dispatcher · Judge · Guardrails

Primitive→specialist tool (deterministic) + zero-context verdict + hard destructive-block guardrails.

`pipeline/pentest-staging/*.py`

L — DVWA Lab (validation)

Isolated RCE target used to prove exploitation end-to-end before trusting agent output.

`docker: dvwa :8082 (localhost only)`

M — Typst Report Builder

Template + build.sh: schema/CVSS/dash/leak gates, redacted twin, JSON companion, CSV tracker.

`pipeline/report-template-v2`

N — Email Delivery

Outlook XOAUTH2 send + IMAP verify. Delivers the client-grade PDF to the client mailbox.

`pipeline/mail-helpers/ email_send.py`

Control & data flow: A → B (you authorize in Telegram; the agent runtime enforces the RoE gate). B → C (everything executes on the lab host). The agent **F** talks to models through **D** (router) or **E** (model gateway). F + G + H + I write results into **J**. **K** reads J to confirm/reject. **M** builds the PDF from J, and **N** delivers it. **L** is the offline proof ground.

LAYER 2 End-to-End Workflow (the "step 1 → step N" path)

1 Authorization Input

Component: A → B

You send **"pentest X, authorized"** in Telegram. the agent runtime enforces the **authorization gate** before any active traffic: pre-authorized targets (outreachdesk.cloud, api., auth.) are whitelisted in memory; third-party targets require an explicit RoE statement. Sets the `AUTHORIZATION` + `ROE` scope vars.

`GATE: no RoE = no active traffic` `OUT: authorized target + scope`

Why first: this is the legal firewall. Passive recon is always safe; port scans / ZAP / fuzzing / RCE agent runs are gated behind this step.

2 Recon / Surface Mapping (incl. nmap)

Component: B + nmap on C

Initial mapping. **nmap** executes on the lab host (the agent's later shell also runs it). The agent runtime adds passive recon via direct `curl` / `dig` .

`nmap -sV -sC` `curl` `dig` `whatweb` `crt.sh` `OpenAPI /openapi.json` `robots/sitemap` `OUT: DNS/headers/tech/port map`

ref: A·B·C · nmap on host = source of the port scan you asked about

3 SPA Crawl & Endpoint Discovery

Component: I

Playwright headless crawl of the JS bundle + each route. Captures forms and every XHR/fetch into the evidence store — closes the SPA blind spot the static scanner misses.

`spa_crawler.py` `headless Chromium` `session_manager.py` `OUT: routes + endpoints → J`

ref: I · validated: 16 routes / 62 exchanges on outreachdesk.cloud

4 Agent Pass — autonomous EXPLOITATION

Component: F (via D or E)

The core testing loop. `run-agent.sh` launches **pentestgpt-agent** detached. The **supervisor** plans; **executor** turns run shell commands at the target — this is **the stage where exploitation is actually attempted** (sqli, cmdi, auth-bypass, RCE). Model routes: claude→gateway /v1/messages (deepseek-v4-flash→mimo), or openai-chat→gateway /v1/chat/completions (hy3 / ox-alpha-free).

`pentestgpt-agent` `executor shell → nmap/web probes/sqli` `D: router` `E: model gateway` `OUT: proposed findings + traces`

Halt-on-WAF: the goal text tells the agent to stop & document at a Cloudflare/captcha challenge — it never bypasses. Budget-clamped in `agents.py`.

5 Exploit-DB Correlation (version → known PoC)

Component: searchsploit / exploit-db clone

This is the **Exploit-DB database step** you remembered. After recon yields a service **banner/version** (Stage 2) and the agent/scanners confirm a primitive, the version is correlated against the local **exploit-db** clone: `banner → version → exploit-db → documented negative OR proven PoC`. A matched CVE/PoC feeds the finding into the Judge (Stage 7) and the report's exploit-db chain / negative-testing matrix.

`searchsploit` `git clone gitlab exploitdb` `version fingerprint (nuclei/whatweb)` `OUT: CVE/PoC match → J (finding evidence)`

Pitfall we hit: the GitHub mirror 404s (asks for credentials) — clone from `gitlab.com/exploit-database/exploitdb` instead. Match must be exact version, not just product name.

6 Scanner Pass (web scan + tool confirmation)

Component: G + H

Deterministic scanner confirmation of what the agent proposed. ZAP (bounded, behind mitmproxy to survive Cloudflare TLS) does the **web scan**; nuclei templates fingerprints; sqlmap runs on confirmed SQLi primitives inside the isolated staging net with an auto-cookie.

`ZAP :8092` `nuclei` `sqlmap` `mitmproxy :8083` `OUT: tool_runs → J (evidence_links)`

ref: G·H · "web scan" you asked about = this ZAP/nuclei pass, plus the agent's own in-turn web requests (Stage 4)

7 Verify + Evidence (the Judge)

Component: K reads J

Dispatcher maps each primitive → the right tool (guardrailed). The **Judge** then does a zero-context verdict: deterministic marker scan ("is vulnerable", "uid=") then an LLM verdict (deepseek via gateway) on the raw evidence tail only — never the agent's narrative. Guardrails hard-block destructive categories unless the target is the staging lab.

`dispatcher.py` `guardrails.py` `judge.py (LLM)` `OUT: confirmed / rejected findings in J`

Trust rule: agent output is corroboration, not truth. A finding is only "verified" after the Judge sees reproduced evidence in J.

7 Report Build (Typst)

Component: M

Findings from J → Typst template. Build gates run automatically: schema + refs + CVSS before compile; leak-grep (0 internal tool names/paths/IPs) + dash-gate (0 em/en dashes) + overflow after. Produces the redacted twin for external review.

`build.sh` `validate.py` `redact.py` `tracker_csv.py` `OUT: PDF + JSON companion + CSV`

ref: M · client-grade 20–35pp; report v3.0 all gates green

9 Delivery (email)

Component: N

Files staged to the lab host, then sent via Outlook XOAUTH2. Delivery is verified by reading the mailbox back: subject + per-attachment filename / MIME / decoded size must match.

`email_send.py` `imap_check.py` `OUT: delivered + verified client email`

Validation Loop (offline proof) — DVWA Lab

Component: L

10

Before trusting any new backend/model or reporting a capstone RCE, the agent is pointed at the isolated DVWA lab (`:8082`). Success = a raw command's output echoed back from the target. This is how ox-alpha-free / hy3 were proven to drive a full DVWA RCE.

`dvwa :8082` `set security=low` `watch docker logs` `OUT: proof-of-breach evidence`

KEY Cross-Reference & Caveats

A–E are the control plane: you authorize (A), the agent runtime orchestrates (B) on the lab host (C), models come from the router (D) / model gateway (E).

F is the only component that executes live exploitation. Everything else feeds it evidence (G,H,I) or verifies it (K,J).

Exploitation checkpoints: attempted in Stage 4 (agent executor), Exploit-DB correlated in Stage 5, confirmed in Stage 6 (scanners), judged in Stage 7, proven offline in Stage 10.

nmap ownership: port scan runs on the lab host — both in Step 2 recon and again inside the agent's shell in Step 4.

Model routing pitfall: hy3 / ox-alpha-free are chat-completions ONLY → must use `--backend openai-chat`. claude backend → gateway Anthropic path.

Trust boundary: agent output is corroboration, not truth. A finding is "verified" only after the Judge (K) sees reproduced evidence in J.

Generated from the live skill tree (agentic-pentest-pipeline v3.1.0, pentest-llm-backend, pentestgpt-model-ops, pentestgpt-backend-adapter) + host `docker ps` on lab-host.local. Diagram reflects the current adopted pipeline; component letters are stable cross-references.